

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Отчёт
По лабораторной работе N4
по дисциплине Информационная Безопасность
Вариант: -

Работу выполнил:
Молчанов Федор Денисович Р3413

Работу принял:
Маркина Татьяна Анатольевна

Оглавление

Сканирование	2
Анализ находок	2
1. SQL-инъекция	2
2. Отражённый XSS	2
3. Отсутствие защиты от clickjacking	3
Приоритеты	6
Контрольные вопросы	6
Источники	6

Сканирование

Стенд: DVWA в Docker, <http://127.0.0.1:4280/>, уровень low. **Инструмент:** ZAP 2.17.0, Arch Linux; дата — 07.10.2026.

Через браузер ZAP создана база и выполнен обход разделов. Quick Scan завершился, но SQLi и XSS появились после отправки форм с `id=1` и `name=test` и отдельных Active Scan этих запросов (495 и 309 запросов). Ниже разобраны три типа предупреждений только для DVWA.

Анализ находок

1. SQL-инъекция

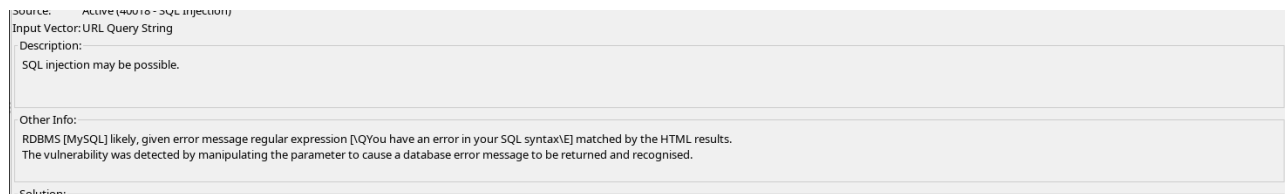
Оценка ZAP: риск — высокий уверенность — средняя.

Что обнаружено. В `/vulnerabilities/sql/` передана кавычка в параметре `id`. Сервер вернул `Uncaught mysqli_sql_exception` и `You have an error in your SQL syntax`. Ошибка воспроизведена вручную в браузере.

Почему опасно. Ответ раскрывает СУБД MariaDB, вызов `mysqli_query`, путь к обработчику и строку 11. Это помогает понять устройство приложения и подобрать дальнейшие проверки. Путь сам по себе не даёт доступ к файлу. Главная угроза — возможность изменить смысл SQL-запроса: расширить выборку и получить чужие записи. Изменение данных или обход входа зависят от запроса и прав пользователя БД.

Вывод. Ошибка — сильный признак SQLi и доказанная утечка диагностических сведений. Чтение и изменение данных через SQLi в работе не проверялись. Приоритет исправления высокий.

Исправление. Параметризованные запросы, проверка типа `id`, минимальные права БД. Подробные ошибки сохранять в журнале [1].



SQL Injection — MySQL: параметр `id`, тестовая кавычка и Evidence.

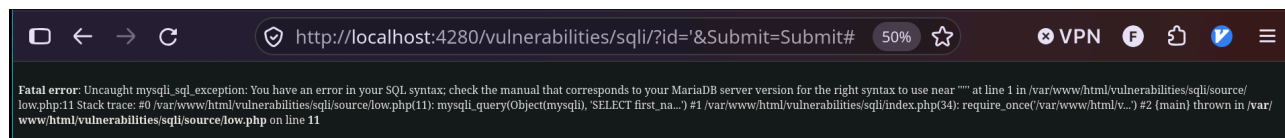


Рис. 1. Та же SQL-ошибка при ручной проверке.

2. Отражённый XSS

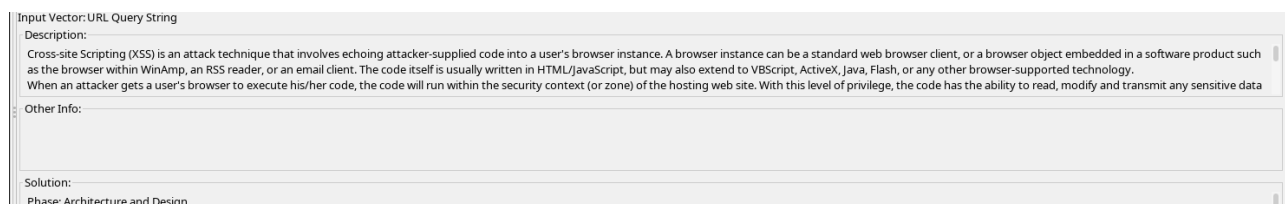
Оценка ZAP: риск — высокий уверенность — средняя; исполнение проверено вручную.

Что обнаружено. В параметр `name` страницы `/vulnerabilities/xss_r/` передана строка `</pre><script>alert(1);</script><pre>`. ZAP обнаружил её отражение; при открытии ссылки браузер показал диалог 1.

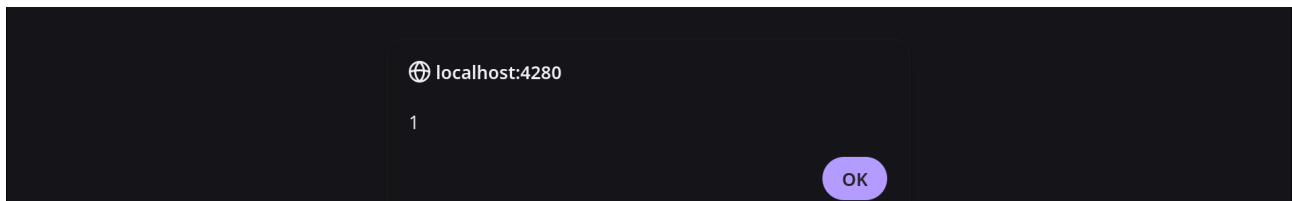
Почему опасно. Сам `alert` безвреден, но доказывает, что ввод стал исполняемым JavaScript. Вместо него код мог бы читать данные страницы, подменять формы и отправлять запросы с полномочиями пользователя. Для атаки нужна жертва, открывшая подготовленную ссылку. `HttpOnly` ограничивает чтение `cookie`, но не мешает коду менять страницу и выполнять запросы.

Вывод. Выполнение JavaScript подтверждено. Потенциальный ущерб высокий, но доступ к ОС, другим сайтам или всем данным сервера из этого не следует.

Исправление. Кодировать вывод согласно HTML-контексту; вставлять обычный текст без интерпретации HTML. CSP использовать как дополнительную защиту [2].



Cross Site Scripting (Reflected): тестовая строка отражена в Evidence.



Ручная проверка: браузер исполнил alert(1) на localhost:4280.

3. Отсутствие защиты от clickjacking

Оценка ZAP: риск — средний уверенность — средняя.

Что обнаружено. Для /setup.php ZAP сообщает Missing Anti-clickjacking Header: отсутствуют X-Frame-Options и защита через CSP frame-ancestors. Это пассивная проверка заголовков, поэтому тестовой строки в Attack нет.

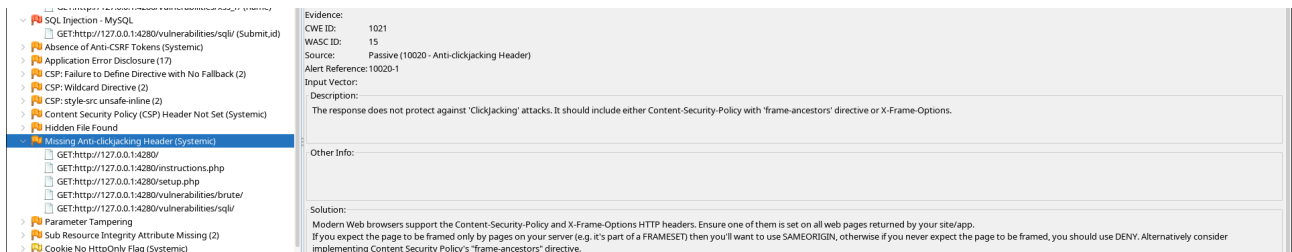
Почему опасно. Если страницу можно встроить в iframe, злоумышленник может скрыть её под обманным интерфейсом. Пользователь думает, что нажимает одну кнопку, а фактически нажимает кнопку приложения. На странице настройки DVWA такое действие — сброс базы.

Ручная проверка. В гостевой книге DVWA сохранена контрольная запись clickjacking-check-02 от имени test. Затем на странице <http://localhost:8000/clickjacking-check.html> в iframe открыта <http://127.0.0.1:4280/setup.php>. Внешняя страница и DVWA имеют разные origin. Кнопка Create / Reset Database совмещена с приманкой «Забрать подарок»: сначала при частичной прозрачности для проверки положения, затем iframe сделан полностью прозрачным. Клик по приманке попал на настоящую кнопку внутри iframe и отправил форму DVWA.

После клика DVWA показала Database has been created. и Setup successful!, а также сообщения о создании таблиц и заполнении исходными данными. При последующем открытии гостевой книги контрольная запись отсутствовала; осталась стандартная запись This is a test comment.. Это подтверждает сброс к исходному состоянию, а не полное опустошение базы.

Вывод. На учебном стенде вручную воспроизведён clickjacking со сбросом БД. Подтверждены встраивание с другого origin, подмена видимого интерфейса и потеря контрольной записи после клика. В конфигурации стенда включено DISABLE_AUTHENTICATION: "true": использование авторизованной сессии жертвы в этом опыте не проверялось. Ущерб ограничен данными учебного стенда.

Исправление. Content-Security-Policy: frame-ancestors 'none' либо X-Frame-Options: DENY, если встраивание не требуется [3].



Missing Anti-clickjacking Header для setup.php.

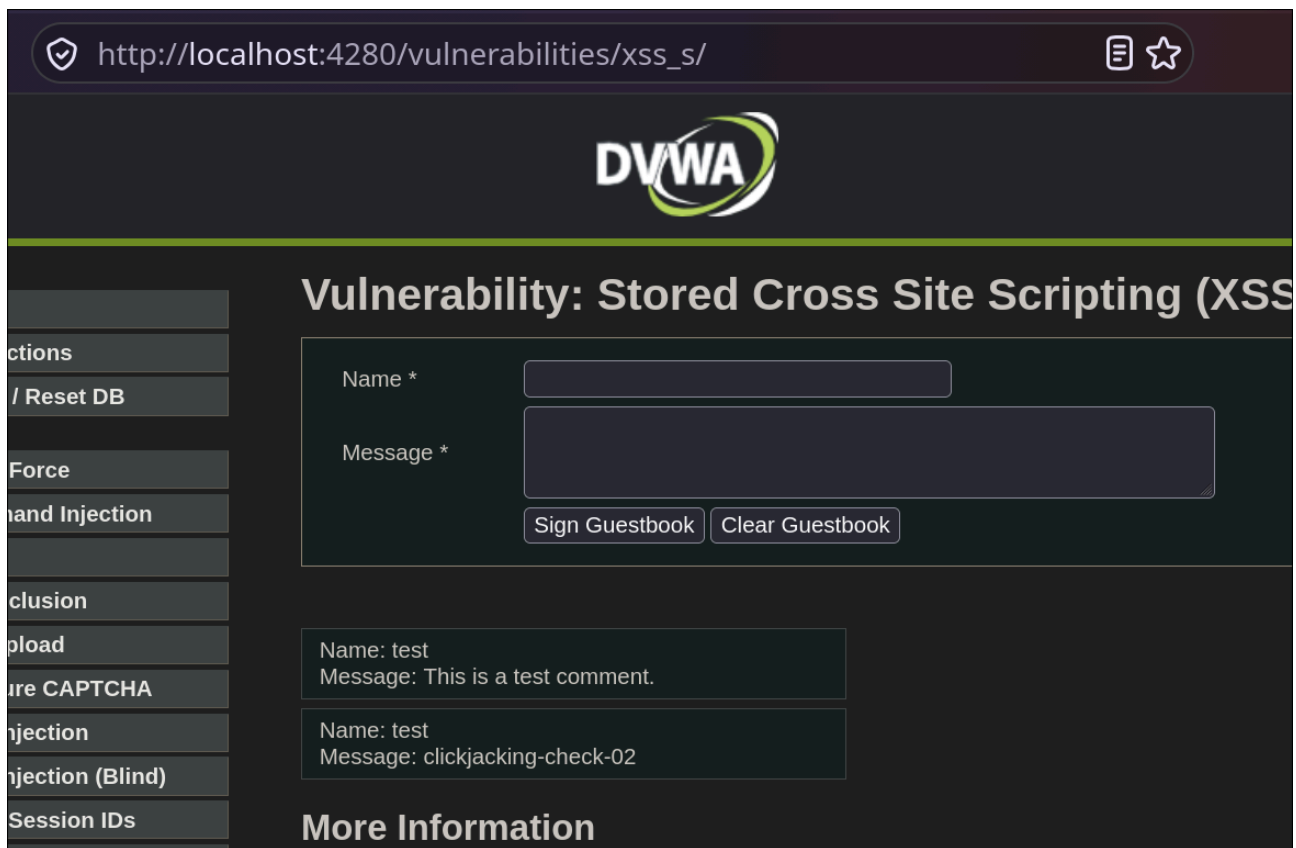


Рис. 2. До опыта: в гостевой книге сохранена контрольная запись clickjacking-check-02.

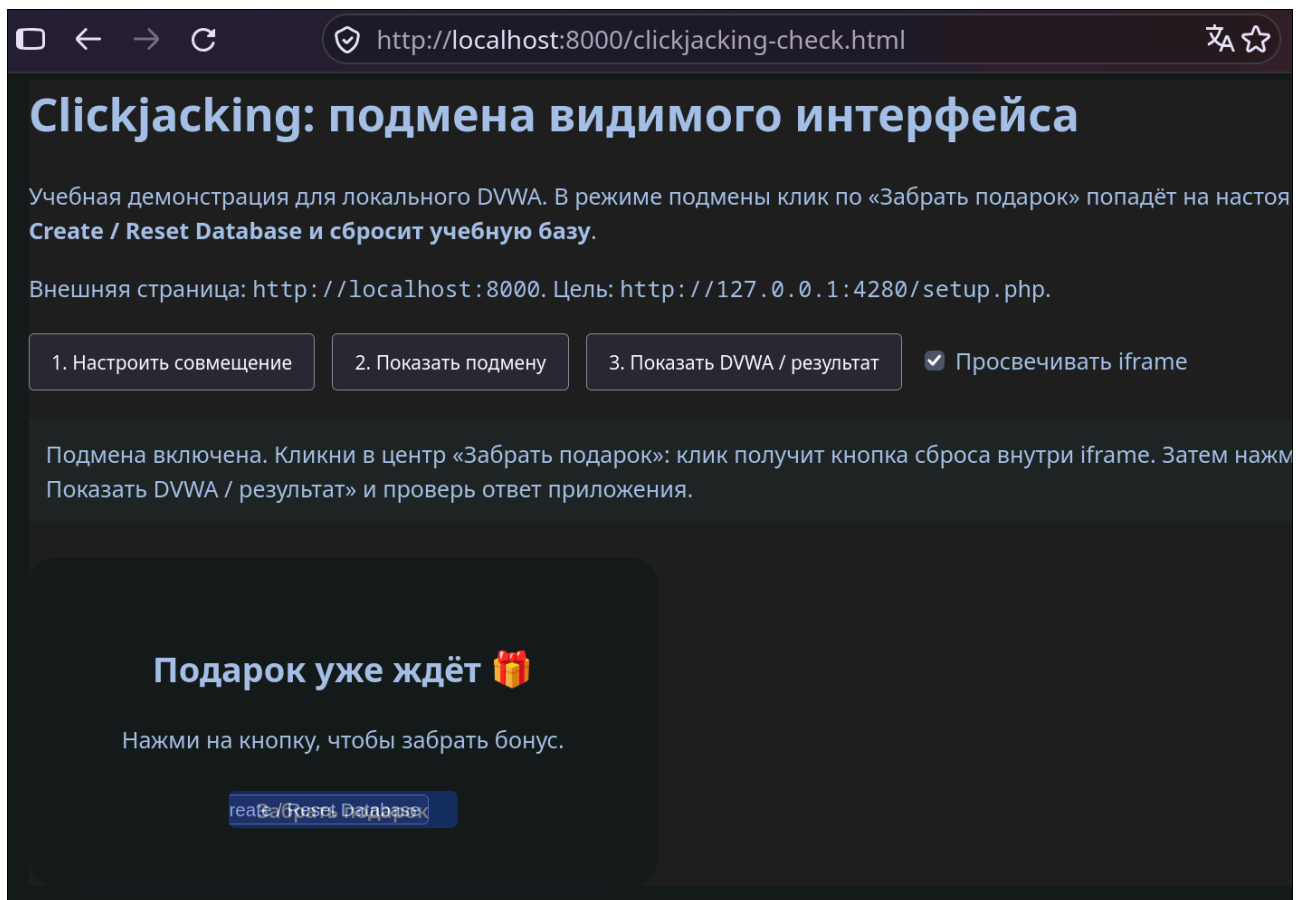


Рис. 3. Настройка совмещения: сквозь iframe видна настоящая кнопка Create / Reset Database поверх приманки.

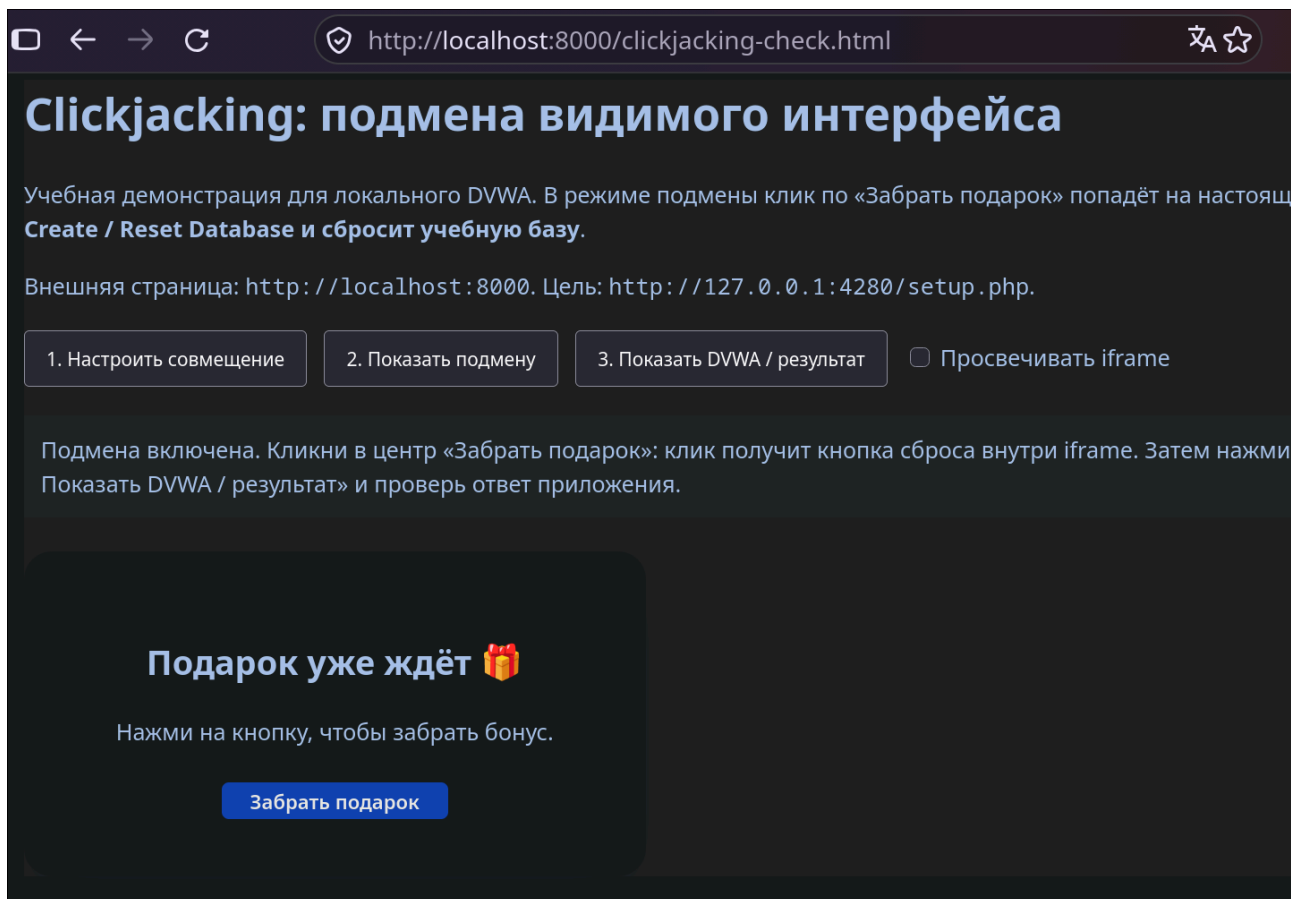


Рис. 4. Подмена включена: iframe полностью прозрачен, пользователю видна кнопка «Забрать подарок».

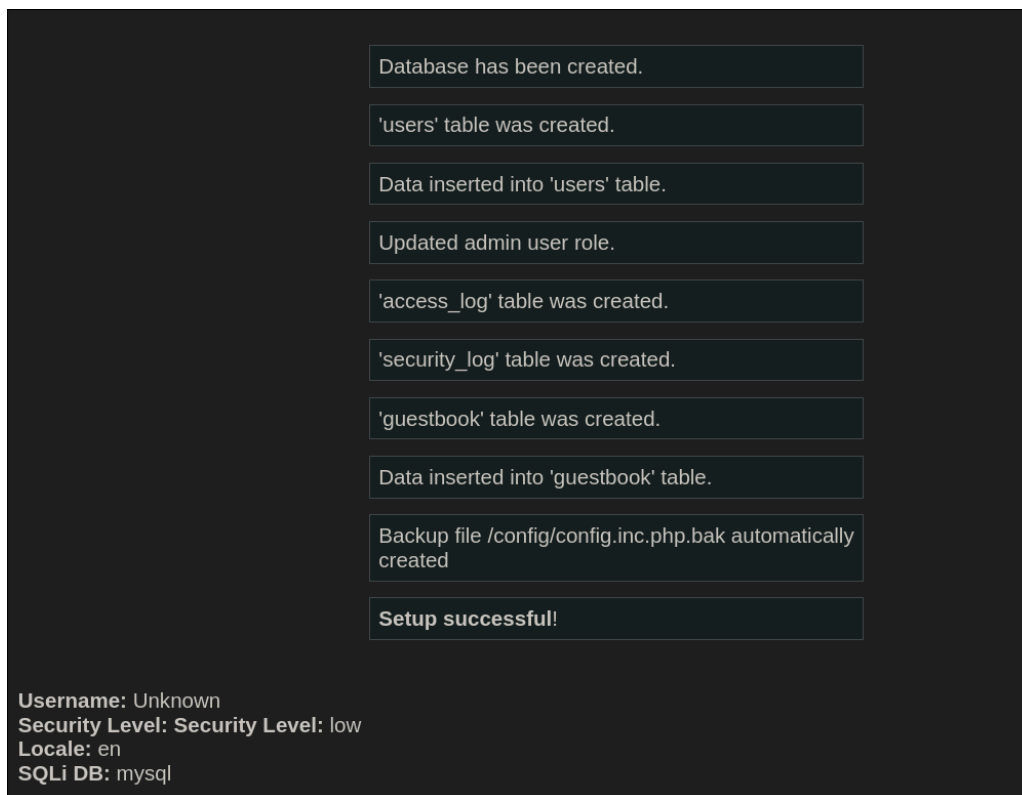


Рис. 5. Ответ DVWA после клика: база и таблицы созданы заново, Setup successful!

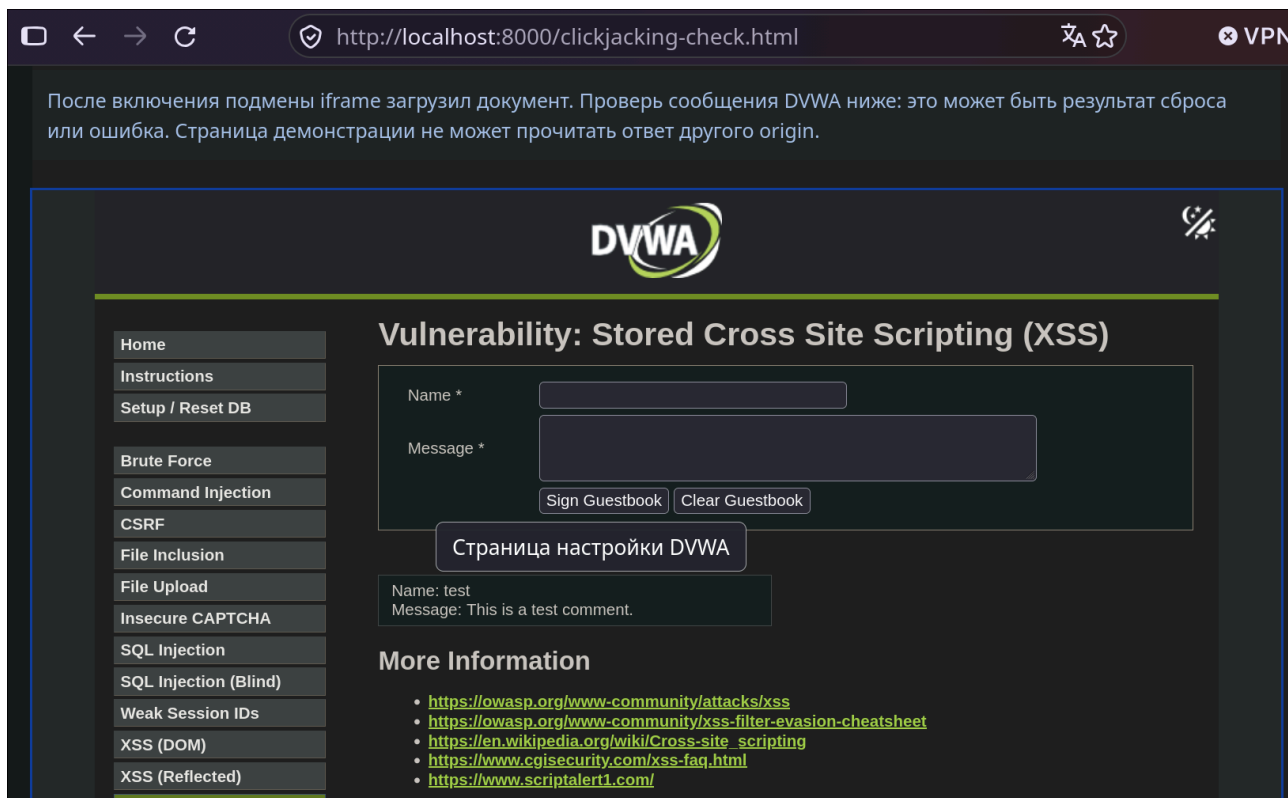


Рис. 6. После сброса: контрольная запись исчезла, в гостевой книге осталась стандартная запись.

Приоритеты

Находка	Оценка по результатам работы
SQLi	Высокий приоритет: воспроизводимая SQL-ошибка; доступ к данным не проверен.
Reflected XSS	Высокий приоритет: исполнение JavaScript подтверждено.
Clickjacking	Средний риск по ZAP; вручную подтверждён сброс учебной БД через подмену интерфейса.

Дополнительный DOM Based XSS оставлен для проверки: в предоставленных подробностях Evidence пусто, механизм обработки данных в DOM не установлен. Он не включён в число подтверждённых находок.

Контрольные вопросы

DAST и SAST. DAST проверяет работающее приложение запросами; SAST анализирует исходный код без запуска.

Последствия XSS. Подмена страницы, чтение доступных данных и действия в приложении с полномочиями пользователя.

Тестирование коммерческих сайтов. Активные проверки требуют разрешения владельца и согласованных границ: они могут нарушить работу и данные.

Источники

- [1] [OWASP: предотвращение SQLi.](#)
- [2] [OWASP: предотвращение XSS.](#)
- [3] [ZAP: Missing Anti-clickjacking Header.](#)